PRODUCT REQUIREMENTS DOCUMENT

Cinepool Jaipur

Product scope, roles, journeys, acceptance criteria and a clearly separated roadmap

Version 1.0 | 28 October 2026

Source baseline: master at bea5ca0

Current interface name: Picturehouse Jaipur

Prepared from the implementation and the documented local MySQL inspection. Current behavior and proposed work are identified separately.

1. Purpose and evidence boundary

Cinepool Jaipur is the project name used for this submission. The running interface and Java package still use **Picturehouse Jaipur**. This PRD describes the product that the repository implements at the stated baseline. A requirement marked **Implemented** is supported by the cited source. **Conditional** means the code exists but a provider or local setting must be configured. **Proposed** means it is a product improvement, not a claim about current behavior. The MySQL schema observations come from the project report's local database inspection on 27 October 2026; data and counts can change later.

The primary evidence is `src/`, `backend/src/main/`, `package.json`, `backend/pom.xml`, and `backend/src/main/resources/application.yml`. The existing `Picturehouse_Jaipur_PRD.docx` and `Picturehouse_Jaipur_SRS.docx` predate this baseline and contain aspirational features; use this pair for the current submission.

2. Product summary

The application lets a Jaipur customer discover films and showtimes, choose seats, pay through Razorpay **Test Mode**, and retrieve a QR ticket. A separate ride board lets signed-in users offer or request a shared trip to a theatre and chat with the other participant. A theatre partner can request access, and an administrator identified by a configured Clerk user ID can approve it. Approved partners can inspect sales and manage movies and showtimes for their assigned theatres.

This is a development prototype. The catalog may contain seeded examples; the API itself labels the data as development data and asks operators to confirm real theatre details before taking bookings. It is not a verified production ticketing or transport service.

3. Problem, goals, and boundaries

3.1 Problem

Planning a cinema visit involves finding a suitable film, an available show, seats, a payment method, and sometimes a way to travel with other people. The project combines these steps in one Jaipur-focused interface and gives theatre partners a scoped management area.

3.2 Current product goals

- **G1 - Discovery:** show active films and future, on-sale Jaipur shows in an eight-calendar-day window, including the current day.
- **G2 - Booking:** make seat state visible, compute fares on the server, and create a ticket only after verified captured test payment.
- **G3 - Retrieval:** let the signed-in purchaser see their own booking reference, details, and QR code; allow email sending when configured.
- **G4 - Shared travel:** allow users to post rides, request a seat, discuss details privately, and let the ride owner decide.
- **G5 - Partner control:** allow management only for approved theatres and restrict administrator approval to configured Clerk IDs.

No production conversion, latency, uptime, accessibility, or payment success target has been established in the source. Such targets require product approval and measurement before they can be claimed.

3.3 Current scope

The city is fixed to Jaipur. Discovery includes films, theatres with matching future shows, movie-title search, dates, showtimes, and seat selection. Signed-in users can complete test checkout, view tickets, request email, use rides, and request theatre access. Approved owners can add movies to an assigned theatre, update eligible movie details, add shows, reschedule a show and change its format, and view booking totals and gross ticket sales. Administrators can review pending theatre-access requests and approve them.

3.4 Outside the current scope

Production Razorpay payments, payment webhooks, WhatsApp delivery, theatre gate QR scanning/redemption, short-lived seat holds, automatic daily schedule rollover, real venue seat layouts, ride payments, route matching, live vehicle tracking, and customer ticket cancellation/refunds are not implemented as end-to-end product flows. A refund attempt exists for a captured test payment when seats cannot be reserved; that is not a general refund portal.

4. Users and permissions

User	Current need	Current permitted behavior
Visitor	Explore films and showtimes	Read catalog and seat availability; the ride GET endpoint is public, although the ride page requires Clerk configuration to render.
Signed-in customer	Book and coordinate travel	Create a test order, verify payment, view own tickets, post rides, request to join, and chat with ride participants.
Theatre applicant	Become a partner	Request access to a listed theatre and see request status. A request alone grants no management access.
Approved theatre owner	Manage assigned venue	View sales and manage movies/showtimes for approved theatre IDs only.
Configured administrator	Grant partner access	Read pending access requests and approve them using a Clerk subject listed in `ADMIN_CLERK_USER_IDS`.

The same Clerk account can perform customer actions and hold one or more approved theatre memberships. There is no separate pre-created Gmail login for a theatre in this baseline.

5. Core journeys and acceptance criteria

PRD-01 - Discover a show (Implemented)

Journey: open the Movies view; see active films; choose a film or theatre; select today or one of the next seven dates; choose a future show. The API uses Asia/Kolkata time, excludes shows earlier than now, and returns on-sale shows through the end of day seven. The interface refreshes catalog data every 15 seconds and on tab focus/visibility. Movie search matches titles; the placeholder also mentions theatres, but theatre-name search is not implemented.

Acceptance criteria: a show earlier than the API's current time is absent; on-sale future shows at active Jaipur theatres appear under the selected date; an empty date/movie combination states that no show is available.

PRD-02 - Select seats (Implemented)

Journey: choose a show and inspect its seat map. The current map is fixed at 48 seats, A1-F8. Rows A-C cost INR 250, D-E INR 300, and F INR 500 per seat. The modal refreshes seat flags every 2.5 seconds and removes newly booked seats from the current selection.

Acceptance criteria: booked seats cannot be selected in the interface; the total changes as seats are selected; the backend rejects invalid, duplicate, unavailable, or more than eight requested seats. The server remains the authority at payment verification.

PRD-03 - Pay and create a ticket (Conditional)

Journey: a signed-in customer asks the backend to create a Razorpay test order for selected seats. The backend calculates the amount in paise, records the order, and the frontend opens Razorpay Checkout. After checkout, the backend checks the signature and fetches the provider payment, requiring the saved order, amount, INR currency, and captured status to match. A database transaction locks the selected seat rows; a successful booking marks them booked and creates a ticket. If the captured payment cannot reserve seats, the backend attempts a full refund and returns a refunded status, or an error requiring support if that attempt fails.

Acceptance criteria: the client cannot dictate the payable amount; a wrong signature or non-captured payment creates no ticket; a second verification for an existing booking returns the same booking; a conflicting seat does not produce a second ticket. Razorpay test keys and Clerk configuration are required.

PRD-04 - View and email tickets (Conditional)

Journey: the purchaser opens My tickets and sees their booking reference, movie, theatre, show, seats, amount, and generated QR. The purchaser can request the ticket email again. Sending needs SMTP settings and an `email` claim in the Clerk token.

Acceptance criteria: one account cannot retrieve another account's ticket through these endpoints; the QR represents a generated ticket code; successful email sending updates the sent marker. The marker is not independent proof of inbox delivery. Scanning/redeeming the QR at a gate is not implemented.

PRD-05 - Share a ride (Implemented)

Journey: browse rides for a date; sign in to post a ride with pickup area, theatre name, movie title, free-text showtime, departure, available seats, and contribution; request to join another person's ride; exchange messages; the ride owner accepts or declines. Conversation endpoints are limited to the two participants. The application does not check for a movie ticket before posting or requesting a ride.

Acceptance criteria: departure must be in the future on the selected date, no more than 90 days ahead; seat count is 1-6; contribution is INR 1-10,000; a user cannot request their own ride; only the ride owner may decide; an accepted request reduces capacity once; a declined conversation is closed to new messages.

PRD-06 - Request and manage theatre access (Implemented)

Journey: a signed-in applicant selects a theatre and requests access. The request remains pending until a configured administrator approves it. An approved owner sees their assigned theatres, shows, booking counts and gross sales, then uses forms to add or update catalogue items and shows.

Acceptance criteria: an unapproved applicant cannot access the dashboard or write to a theatre; an approved owner cannot write to a different theatre; only a configured administrator can approve; shared movie details cannot be edited by one owner when that movie is used by another theatre. A new show generates the fixed 48-seat inventory.

PRD-07 - Basic presentation controls (Implemented)

The interface offers a light/dark toggle and responsive layouts. The theme is held in React state, so it resets when the page reloads. The top search field filters movie titles. These behaviors should be described to users accurately.

6. Product rules and dependencies

- Catalog time filtering is performed by the backend in `Asia/Kolkata`; the frontend date list is built when the app mounts. A tab left open across midnight may need a reload to rebuild date options.
- New showtimes must be after the backend's current local time and no more than 90 days ahead. The update endpoint checks future time but currently does not apply the same 90-day limit.
- The show editor saves time and format. Although its request includes a screen and movie, the current update method does not change those associations.
- Ride text fields store theatre, movie, and showtime labels rather than foreign keys. The ride board does not prove that a ride matches a current catalog show.
- Clerk provides sign-in; the API validates signed JWTs and uses the token subject for account ownership. SMTP and Razorpay Test Mode are separate optional configurations.
- The backend defaults to the MySQL `picturehouse` database and uses JPA schema update. An existing local MySQL instance and credentials are required.

7. Release-readiness assessment and proposed roadmap

The repository supports a local demonstration and a test-mode payment flow. Before a real launch, the following are **proposals**, not current functions:

1. Replace seed examples with theatre-approved catalog, real screens and seating plans; establish a controlled scheduling process.
2. Add payment webhooks, reconciliation, idempotent event handling, customer support and general refund/cancellation operations for production payments.
3. Add a seat hold with expiry or another explicit inventory policy so users know when a seat is reserved during checkout.
4. Add gate QR verification/redemption with staff authorization and an auditable check-in state.
5. Make ride posts refer to real showtime IDs, verify ticket eligibility if required, and add moderation/reporting and safety controls.
6. Correct the movie search placeholder or implement theatre search; refresh date choices across midnight; reconcile the show editor with the fields it presents.
7. Replace `ddl-auto: update` with reviewed database migrations before production and define backup/restore procedures.

8. Evidence map

Area	Source of current behavior
Catalog/date/seat API	`backend/src/main/java/in/picturehouse/jaipur/api/CatalogController.java`
Frontend views and refresh	`src/App.jsx`, `src/RideBoard.jsx`, `src/OwnerPortal.jsx`
Test checkout and seat locking	`backend/src/main/java/in/picturehouse/jaipur/api/RazorpayController.java`, `backend/src/main/java/in/picturehouse/jaipur/repository/ShowtimeSeatRepository.java`
Tickets and email	`backend/src/main/java/in/picturehouse/jaipur/api/TicketController.java`, `backend/src/main/java/in/picturehouse/jaipur/integration/TicketEmailService.java`
Rides	`backend/src/main/java/in/picturehouse/jaipur/api/RideController.java`
Owner/admin access	`backend/src/main/java/in/picturehouse/jaipur/api/OwnerPortalController.java`
Authentication and configuration	`backend/src/main/java/in/picturehouse/jaipur/config/SecurityConfig.java`, `backend/src/main/resources/application.yml`

Document control: This PRD is a product description of the observed code baseline, not a promise that unconfigured integrations work. The companion SRS gives precise interface, data, security and acceptance details.